

PROGRAMA DE ESTUDIO

Administración de Centros de Cómputo

MÓDULO

Gestión de Plataformas y Servicios de TI.



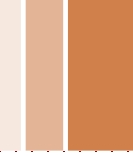
INSTITUTO DE
EDUCACIÓN SUPERIOR
TECNOLÓGICO PÚBLICO

"PAIJÁN"

UNIDAD DIDÁCTICA

Administración de Plataformas de TI

- Llegar puntualmente al laboratorio de cómputo e ingresar con orden.
- Mantener un ambiente de respeto y colaboración con los compañeros de equipo.
- Usar los equipos y el software exclusivamente para las actividades de la sesión.
- Levantar la mano y esperar el turno para participar o consultar dudas.
- Cuidar los equipos, materiales y el orden del laboratorio durante toda la práctica.



Fecha: 28, 29 y 30 de septiembre de 2026

Docente: Ing. Carlos Abilio, Angulo Zegarra

- ¿Qué diferencia recuerdan entre una máquina virtual y un contenedor de la práctica anterior?
- ¿Qué es un archivo YAML y dónde se usan este tipo de archivos en tecnología?
- ¿Cómo creen que se comunican entre sí varios contenedores dentro de un mismo servidor?

Video: “Docker Compose desde cero: la guía completa con ejemplos prácticos”

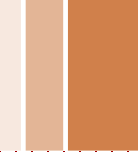
<https://www.youtube.com/watch?v=cCcLAdUUBjg>

Video: [“Cómo Instalar Docker en Debian Paso a Paso”](#)



Una aplicación empresarial está formada por un servidor web, una aplicación y una base de datos que deben funcionar juntos. Instalar y conectar cada componente manualmente en cada servidor toma horas, genera errores de configuración y hace casi imposible reproducir el mismo despliegue en otro servidor. ¿Existe una forma de definir los tres servicios en un solo archivo y levantarlos todos con un único comando?

Al concluir la sesión, los estudiantes despliegan una aplicación multicontenedor utilizando Docker Compose: definen los servicios Nginx, aplicación y base de datos en un archivo `compose.yaml`, configuran su red y el volumen de persistencia de datos, y verifican su correcto funcionamiento con los comandos `docker compose up`, `ps`, `logs` y `down`.



Marco conceptual

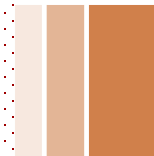
- **Docker Compose:** herramienta de orquestación de contenedores que define y ejecuta aplicaciones multicontenedor con un solo comando; lee su configuración desde el archivo `compose.yaml`.
- **Archivo `compose.yaml`:** archivo YAML donde se declaran servicios, redes y volúmenes; su estructura jerárquica depende de la sangría (indentación) del texto.
- **Servicios:** cada contenedor de la aplicación (web con Nginx, aplicación y base de datos) declara su imagen, puertos y variables de entorno.
- **Redes:** Compose crea una red propia donde los contenedores se comunican entre sí usando el nombre del servicio como host.
- **Volúmenes:** persisten los datos de la base de datos y de la aplicación fuera del ciclo de vida de los contenedores.

EJEMPLO PRÁCTICO: ARCHIVO COMPOSE.YAML

Creación del archivo compose.yaml con los servicios Nginx, aplicación y base de datos, su red y el volumen de persistencia.

```
carlos@debian13-apti: ~/app-compose

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
  app:
    image: miapp:1.0
    volumes:
      - datos_app:/var/www/app
  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: admin2026
    volumes:
      - datos_db:/var/lib/mysql
volumes:
  datos_app:
  datos_db:
```



Levantamiento de la aplicación multicontenedor con un solo comando: `docker compose up -d`.

```
carlos@debian13-apti: ~
```

```
$ docker compose up -d
```

```
[+] Running 4/4
```

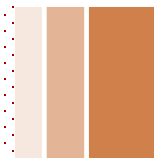
```
Network app-compose_default    Created
```

```
Volume "app-compose_datos_db"  Created
```

```
Container app-compose-db-1     Started
```

```
Container app-compose-app-1     Started
```

```
Container app-compose-web-1     Started
```



Verificación del estado de los servicios y revisión de los registros con docker compose ps y logs.

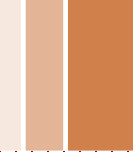
```
carlos@debian13-apti: ~  
  
$ docker compose ps  
NAME                IMAGE             STATUS             PORTS  
app-compose-web-1   nginx:latest      Up 2 minutes      0.0.0.0:8080->80/tcp  
app-compose-app-1   miapp:1.0         Up 2 minutes  
app-compose-db-1    mariadb:11        Up 2 minutes      3306/tcp  
  
$ docker compose logs web  
web-1 | nginx ready - start worker processes
```

Comprobación de la persistencia de los datos y cierre de la aplicación con docker compose down.

```
carlos@debian13-apti: ~  
  
$ docker compose exec db mariadb -uroot -padmin2026 -e "CREATE DATABASE prueba;"  
$ docker compose down  
[+] Running 4/4  
Container app-compose-web-1      Removed  
Container app-compose-app-1      Removed  
Container app-compose-db-1       Removed  
Network app-compose_default      Removed  
$ docker volume ls  
local    app-compose_datos_db  
local    app-compose_datos_app
```

1. Formar equipos de 3 a 4 estudiantes en el laboratorio de cómputo.
2. Ingresar al servidor Debian 13 y verificar Docker Engine y el plugin Compose (docker compose version).
3. Preparar la carpeta del proyecto y escribir el archivo compose.yaml con los servicios Nginx, aplicación y base de datos, su red y volumen de persistencia.
4. Levantar la aplicación con docker compose up -d; verificar los servicios (docker compose ps) y revisar los registros (docker compose logs).
5. Comprobar la persistencia de los datos con docker compose down y docker volume ls.
6. Registrar evidencias (capturas de pantalla) de cada paso y elaborar el informe con el archivo compose.yaml y las evidencias, usando la IA de forma responsable y verificada (guía de práctica S7).

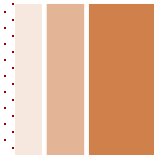
Criterio	Descriptor	Puntaje
Conceptual	Explica Docker Compose, la estructura del archivo YAML, servicios, redes y volúmenes	4
Procedimental	Despliega con Docker Compose una aplicación multicontenedor (Nginx, aplicación y base de datos) funcionando correctamente	8
Actitudinal	Trabaja colaborativamente con responsabilidad, entrega evidencias completas y usa la IA de forma verificada	8
Total		20



- Docker Compose orquesta aplicaciones multicontenedor definidas en un solo archivo YAML.
- En compose.yaml cada servicio declara su imagen, puertos y variables de entorno.
- Los contenedores se comunican por el nombre del servicio dentro de la red creada por Compose.
- Los volúmenes garantizan que los datos persistan aunque los contenedores se eliminen.

- ¿Qué ventajas ofrece Docker Compose frente a ejecutar contenedores de forma individual?
- ¿Por qué es importante el volumen de persistencia para la base de datos de la aplicación?
- ¿En qué situaciones de su práctica profesional aplicarían aplicaciones multicontenedor?

- Docker Docs. (2026). Docker Compose overview. <https://docs.docker.com/compose/>
- Docker Docs. (2026). Compose file reference. <https://docs.docker.com/reference/compose-file/>
- Campo, R., Del Campo, M., y Raya, V. (2014). Gestión de proyectos (1.ª ed.). RAMA.
- Martínez, A., Morris, E., y Lossio, F. (2016). La gestión de proyectos en el Perú (1.ª ed.). Esan Ediciones.



*Muchas
Gracias!*